

## CO2 ASSESSMENT 2

**Q1 . Apply relational algebra operations ( $\sigma$ ,  $\pi$ ,  $\cup$ ,  $-$ ,  $\times$ ,  $\bowtie$ ) on the given Employee and Department relations to retrieve required records**

**SOLUTION :**

- Start by writing out the schema: Employee(EmpID, EName, Salary, DeptID), Department(DeptID, DName, Location).
- $\sigma$  (selection) = "filter rows":  $\sigma$  condition (Relation) — e.g.  $\sigma$  Salary>50000(Employee).
- $\pi$  (projection) = "pick columns":  $\pi$  col1,col2(Relation).
- You can nest them:  $\pi$  EName( $\sigma$  Salary>50000(Employee)) means "filter first, then pick columns."
- $\cup$  and  $-$  only work on two relations with the *same columns* (union-compatible) — used to combine or subtract two filtered sets.
- $\times$  pairs every row of one relation with every row of another (gets messy fast).
- $\bowtie$  (natural join) is  $\times$  plus automatically matching on the shared column (like DeptID) — this is what you use to combine Employee and Department meaningfully.
- Method to answer this kind of question: write one line per required operator, applying it to the given relations directly.

Employee(EmpID, EName, Salary, DeptID)  
Department(DeptID, DName, Location)

$\sigma$  Salary > 50000 (Employee)  
 $\pi$  EName, Salary (  $\sigma$  Salary > 50000 (Employee) )  
 $\pi$  EName, DName ( Employee  $\bowtie$  Department )  
 $\pi$  EmpID( $\sigma$  DeptID='D01'(Employee))  $\cup$   $\pi$  EmpID( $\sigma$  DeptID='D02'(Employee))  
 $\pi$  EmpID( $\sigma$  DeptID='D01'(Employee))  $-$   $\pi$  EmpID( $\sigma$  DeptID='D02'(Employee))  
Employee  $\times$  Department

**Q2. Write SQL DDL statements to create STUDENT(SID, Name, DOB, DeptID) and DEPARTMENT(DeptID, DeptName) tables with all appropriate constraints.**

**SOLUTION :**

- Create tables in dependency order: the table with no foreign key first (DEPARTMENT), then the one referencing it (STUDENT).
- For each column decide: is it unique/identifying? → PRIMARY KEY. Must always have a value? → NOT NULL. Must match another table? → FOREIGN KEY ... REFERENCES. Needs a rule (like a date range)? → CHECK.
- Write CREATE TABLE name (col type constraint, ...); for each table in order  
CREATE TABLE DEPARTMENT (  
    DeptID VARCHAR(10) PRIMARY KEY,  
    DeptName VARCHAR(50) NOT NULL UNIQUE  
);

```
CREATE TABLE STUDENT (  
    SID VARCHAR(10) PRIMARY KEY,  
    Name VARCHAR(50) NOT NULL,  
    DOB DATE NOT NULL CHECK (DOB <= CURRENT_DATE),  
    DeptID VARCHAR(10),  
    CONSTRAINT fk_student_dept FOREIGN KEY (DeptID)  
        REFERENCES DEPARTMENT(DeptID)  
        ON DELETE SET NULL  
        ON UPDATE CASCADE  
);
```

**Q3 .Formulate SQL queries using GROUP BY, HAVING, and aggregate functions (COUNT, SUM, AVG, MAX, MIN) on a given Sales dataset.**

**SOLUTION :**

- Ask: "per what do I want to summarize?" (e.g. per product) → that's your GROUP BY column.
- Ask: "what do I want to compute for each group?" → COUNT, SUM, AVG, MAX, MIN go in the SELECT list alongside the grouped column.
- WHERE filters rows *before* grouping; HAVING filters *groups* after aggregating (use it only when your condition involves an aggregate, like

**Q4. Write a correlated sub-query to find employees earning more than the average salary of their own department**

**SOLUTION :**

.: Give the outer table an alias (e1) and inner copy of the same table another alias (e2).

· Inner query: compute the average salary for "this employee's department" — reference e1.DeptID inside the inner WHERE so it recalculates per row.

· Outer WHERE: e1.Salary > (inner query).

```
SELECT e1.EmpID, e1.ENAME, e1.DeptID, e1.Salary
FROM Employee e1
WHERE e1.Salary > (
    SELECT AVG(e2.Salary)
    FROM Employee e2
    WHERE e2.DeptID = e1.DeptID
);
```

**Q5. Apply all types of JOIN operations (INNER, LEFT OUTER, RIGHT OUTER, FULL OUTER, CROSS) on Employee and Project tables.**

**SOLUTION :**

- Same base pattern for all: SELECT ... FROM Employee e JOIN Project p ON e.ProjID = p.ProjID;
- Just swap the JOIN keyword: JOIN (inner, only matches), LEFT JOIN (all of left table + matches), RIGHT JOIN (all of right table + matches), FULL OUTER JOIN (all of both), CROSS JOIN (no ON clause at all — full pairing).

**Q6. Create a view 'DeptSalaryView' that displays department name, total employees, and average salary. Write a query on this view**

**SOLUTION :**

- Write the SELECT query you'd want to reuse (grouped by department, with COUNT and AVG), wrap it as CREATE VIEW viewname AS <query>;
- Then treat the view name like a table in a normal SELECT afterward.

```
CREATE VIEW DeptSalaryView AS
SELECT d.DeptName,
       COUNT(e.EmpID) AS TotalEmployees,
       AVG(e.Salary) AS AvgSalary
FROM   Department d
JOIN   Employee e ON d.DeptID = e.DeptID
GROUP BY d.DeptName;
```

```
SELECT * FROM DeptSalaryView
WHERE AvgSalary > 60000
ORDER BY AvgSalary DESC;
```

**Q7 . Construct a relational algebra expression for the following: Find names of students who are enrolled in all courses offered**

**SOLUTION :**

Enroll(SID, CID) -- (student, course) pairs actually taken

Course(CID, CName) -- full list of offered courses

**Goal:** Find students who are enrolled in *every* course that's offered — not just some.

**Step 1 — Direct division form**

$\text{Result} = \pi \text{ SID, CID (Enroll)} \div \pi \text{ CID (Course)}$

This says: divide the enrollment pairs by the full set of course IDs. What's left is every SID that appears against **all** course IDs, not just a few.

**Step 2 — Why division works (the intuition)**

- $\pi \text{ SID,CID(Enroll)}$  is a table of every (student, course) pair that really happened.
- $\pi \text{ CID(Course)}$  is the list of all courses that exist.
- $R \div S$  keeps a value of R's other column (SID) only if **that value is paired with every single value in S** in R. So a student only survives the division if they show up next to every course — exactly "enrolled in all courses."

Walk-through:

- $\text{AllStudents} \times \text{AllCourses}$  generates every possible (student, course) pair — i.e., what a student *would* need if they took everything.
- Subtracting the actual Enroll pairs leaves Missing: every (student, course) combination that a student *didn't* take.
- Projecting Missing onto SID gives every student who is missing **at least one course**

**Q8 . Write SQL DML statements (INSERT, UPDATE, DELETE) with**

**SOLUTION:**

INSERT INTO STUDENT (SID, Name, DOB, DeptID) VALUES ('S101', 'Arun Kumar', '2004-05-12', 'D01'); UPDATE STUDENT SET DeptID = 'D02' WHERE SID = 'S101'; DELETE FROM STUDENT WHERE SID = 'S101';

Just write one INSERT, one UPDATE, one DELETE on your STUDENT table, and in words explain what constraint would block a bad value (e.g. inserting a DeptID that doesn't exist in DEPARTMENT gets rejected by the FK constraint).

**Q9 . Apply tuple relational calculus to express a query: Find all employees who work in the 'IT' department and earn more than 50000.**

**SOLUTION:**

- Format is always:  $\{ t \mid t \in \text{Relation} \wedge \text{condition1} \wedge \text{condition2} \}$
- Plug in:  $\{ t \mid t \in \text{Employee} \wedge t.\text{Dept} = \text{'IT'} \wedge t.\text{Salary} > 50000 \}$

$\{ t \mid t \in \text{Employee} \wedge t.\text{Dept} = \text{'IT'} \wedge t.\text{Salary} > 50000 \}$

**Q10..Design a relational schema for an Airline Reservation System and write 5 SQL queries demonstrating joins, sub-queries, and views.**

**SOLUTION:**

- Identify entities (Passenger, Flight) and the relationship between them (Booking, since one passenger can book many flights and vice versa — that's why Booking needs its own table with two foreign keys).
- Then just reuse the join/sub-query/view patterns from Q5, Q4, Q6 against this schema.

```
CREATE TABLE PASSENGER (  
    PassID VARCHAR(10) PRIMARY KEY,  
    PName VARCHAR(50)  
);
```

```
CREATE TABLE FLIGHT (  
    FlightID VARCHAR(10) PRIMARY KEY,  
    Origin VARCHAR(30),  
    Destination VARCHAR(30),  
    Fare DECIMAL(10,2)  
);
```

```
CREATE TABLE BOOKING (  
    BookingID VARCHAR(10) PRIMARY KEY,  
    PassID VARCHAR(10) REFERENCES PASSENGER(PassID),  
    FlightID VARCHAR(10) REFERENCES FLIGHT(FlightID)  
);
```

-- 1) Join: passenger + flight booked

```
SELECT p.PName, f.Origin, f.Destination  
FROM Booking b  
JOIN Passenger p ON b.PassID = p.PassID  
JOIN Flight f ON b.FlightID = f.FlightID;
```

-- 2) Sub-query: passengers on the costliest flight

```
SELECT p.PName FROM Passenger p  
JOIN Booking b ON p.PassID = b.PassID
```

